

바이브 코딩 실습 워크숍

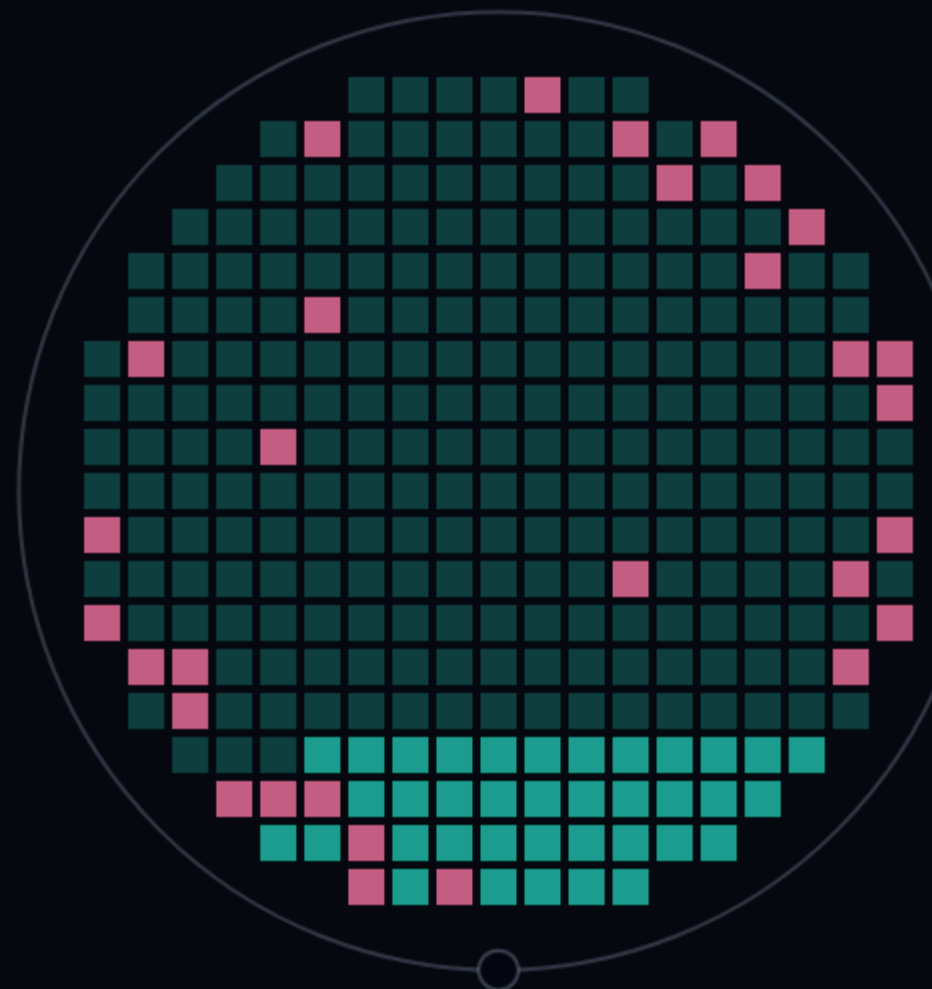
말로 시키고, 눈으로 확인하고, 다시 시킨다.

오늘, **내 업무에 쓸 도구 하나**를 직접 만들어 갑니다.

🕒 90분 · 3시간 · 8시간

🔬 반도체 엔지니어 대상

☞ 코딩 경험 불필요



퇴근할 때, 이것들을 가져갑니다

3+

직접 만든 동작하는 결과물

1

내 업무용 도구 — 마지막 실습에서 만듭니다

1

"내 업무도 되겠는데?"라는 확신

오늘의 진짜 목표는 코딩 실력이 아니라 **자신감**입니다.
"AI에게 시키면 되는구나"를 몸으로 익히는 것.

오늘은 어느 코스인가요?

덱은 하나지만 길이는 셋입니다. 강사가 오늘의 코스를 먼저 알려줍니다 — 각 실습 오른쪽 위 칩(🕒 90 ⬡ 3h 📅 8h)이 "이 실습이 어느 코스에 들어가는지"입니다.

90분

⚡ 스프린트

- 명함 웹페이지 (복붙)
- 고장난 앱 고치기
- 웨이퍼 맵 보스전
- 내 업무 도구

3시간

⬡ 표준

- 스프린트 전부 +
- 룰렛 · 빈칸 실습
- SPC 셀프 검증
- 스킬로 굳히기

기본

8시간

📅 풀데이

- 표준 전부 +
- 부록 실습 21개
- 심화 트랙 (배포까지)
- 미니 프로젝트 확장

세 코스 모두 마지막은 "내 업무 도구 하나"입니다. 쉬운 것으로 손을 풀고, 마지막에 내 일로 옵니다.

실습 형식이 계속 바뀝니다

손을 푸는 앞부분(위밍업·명함·룰렛)은 전부 복불이고, 그 뒤로는 형식이 계속 바뀝니다 — 손이 하는 일이 매번 달라집니다. 다섯 형식 모두 혼자 합니다.



스프린트
SPRINT

프롬프트를 드립니다.
복불로 결과 보기



구조
RESCUE

이미 고장난 것을
살려내기



빈칸
BLANK

프롬프트를 안 드립니다.
직접 쓰기



보스전
BOSS

정답이 있는 과제.
채점기가 점수를 준다



셀프 검증
SELF-CHECK

내 결과를 내가
의심해 보기

순서가 곧 난이도입니다. 복불로 시작해서, 마지막엔 프롬프트를 직접 써서 내 업무 도구를 만듭니다.

오늘의 흐름

—쉬운 복불에서내 업무로

시각	형식	무엇을	결과물
0:00	—	코스 · 규칙 · 보안 약속 · 워밍업	첫 응답 받아보기
0:15	SPRINT	명함 웹페이지 — 프롬프트 복불	브라우저에 뜨는 내 이름
0:30	SPRINT	점심 메뉴 룰렛 — 움직이는 것	돌아가는 돌림판
0:45	RESCUE	고장난 앱 고치기	복구 3단계 경험
1:05	BLANK	측정 CSV 리포트 — 프롬프트 직접	내가 쓴 첫 프롬프트
1:25		휴식	—
1:40	BOSS	🏆 웨이퍼 맵 보스전 — 자동 채점	내 점수
2:05	SELF-CHECK	SPC 관리도 + 스스로 검산하기	의심하는 습관
2:30	MY TOOL	내 업무 도구 — 오늘의 마지막 실습	내일 쓸 도구
2:55	—	스킬로 굳히기 + 마무리	남는 파일 하나

모든 실습은 개인 작업입니다. 막히면 조교를 부르거나 Q&A에 익명으로 남기면 됩니다.

바이브 코딩 = 이 루프가 전부



시킨다

원하는 걸 말로 설명



확인한다

결과를 직접 열어본다



고쳐달라 한다

마음에 들 때까지

- 코드를 **읽지 않아도** 됩니다 — 결과만 판단하면 됩니다.
- 한 번에 완벽할 필요 없습니다 — **대화**가 곧 개발입니다.
- 여러분이 잘하는 것: **뭐가 맞는 결과인지 아는 것**. 그거면 충분합니다.

시작 전 점검

—각자 확인, 안 되면 손 들기



필수

- AI 코딩 도구 실행 확인 (Claude Code / 사내 허용 도구)
- 실습 폴더 만들기: 바탕화면/`vibe-workshop`
- 브라우저에서 HTML 파일 열 수 있는지 확인



도구가 처음이라면

- 설치부터 필요하면 → [0교시: 설치와 첫 실행](#) (30분 가이드)
- 터미널에서 실습 폴더로 이동 후 도구 실행
- 모르면 손 들기 — 조교가 옆에서 같이 봐 드립니다



도구가 잘 되는지 테스트

복사

안녕! 오늘 바이브 코딩 워크숍에 참가했어. 잘 연결됐는지 확인하고 싶으니 짧게 인사해줘.

좋은 지시의 3요소



목표

뭘 만들고 싶은가
"점심 메뉴 룰렛을 만들어줘"



맥락

누가, 어디에 쓰는가
"팀원 8명이 매일 점심에 쓸 거야"



제약

형태·조건·취향
"HTML 파일 하나로, 버튼은 크게"

😞 아쉬운 지시

"룰렛 만들어줘"

😎 좋은 지시

"점심 메뉴 룰렛을 HTML 파일 하나로 만들어줘. 메뉴 8개를 내가 수정할 수 있게 하고, 돌리는 버튼은 크게."

단, 처음부터 완벽한 지시를 쓰려고 고민하지 마세요. 짧게 시작하고 대화로 고치는 게 더 빠릅니다.

오늘의 실습 규칙 4가지



에러는 복붙

빨간 글씨가 나오면 해석하려 하지 말고 **통째로 복사해서 AI에게** 붙여넣으세요. "이 에러 고쳐줘" 한 줄이면 됩니다.



30초 룰

30초 이상 고민되면 그냥 AI에게 물어보세요. "**어떻게 하지?**"도 **프롬프트**가 됩니다.



먼저 직접 써 보기

앞부분은 프롬프트를 **드립니다**(복붙 OK). 뒤로 갈수록 **안 드리는** 실습이 나옵니다 — 그때는 어설퍼도 **내 문장으로** 먼저 보내 보세요.



코드는 안 읽어도 됨

결과물만 보고 판단하세요. 코드가 궁금하면 "**방금 코드 설명해줘**"라고 시키면 됩니다.



시작 전, 딱 하나만 약속합시다

실제 사내 데이터·기밀은 오늘 절대 사용 금지

공정 레시피, 실제 수율/측정 데이터, 고객 정보, 미공개 제품명 — 외부 AI 도구에 넣는 순간 유출입니다.

✓ 오늘 우리가 쓰는 것

- **합성(가짜) 데이터** — AI에게 만들어달라고 시킵니다
- 공개된 물리 공식·일반 지식
- 이 슬라이드가 제공하는 예시 텍스트

🏢 회사에서 쓸 때는

- 사내 보안 정책이 허용한 도구/환경에서만
- 실데이터 전에 합성 데이터로 먼저 검증
- 모르면 보안 담당자에게 먼저 확인

PART 1 · 0:00-0:45

첫 결과물 만들기

쉽고, 눈에 바로 보이는 것부터. **프롬프트는 드립니다 — 복사해서 붙여넣으면 됩니다.**
여기서 만드는 건 장난감이지만, 익히는 근육은 진짜입니다.

🎯 시키기→확인→고치기 루프 체화

⚡ 복불 실습 2개



키보드 풀기용. AI에게 말로 시켜서 텍스트 결과를 받아봅니다. 아직 파일도 안 만듭니다.

📄 프롬프트 ①

복사

오늘 하루 바이브 코딩 워크숍에 참여하는 반도체 엔지니어야. 오늘 나를 위한 응원 문구를 3개만 만들어줘. 반도체 용어를 살짝 섞어서 재치있게.

📄 프롬프트 ② — 고쳐달라 하기

복사

좋아, 그중 두 번째가 마음에 들어. 그 톤으로 5개 더 만들어줘. 이번엔 4행시로.

👁️ 예상 결과

응원 문구 3개만 만들어줘...

"오늘도 무결점, 수율 100%의 하루 되세요!" ✨

두 번째가 마음에 들어. 4행시로 5개 더!

✅ 완료 조건

✓ 답변을 받았다

✓ 이어서 수정 요청을 한 번 해봤다

생애 첫 웹페이지 — 파일 하나가 만들어지고, 브라우저에서 열립니다.

아래 프롬프트를 그대로 복사해서 붙여넣기만 하면 됩니다. 오늘 가장 쉬운 실습입니다.

📋 시작 프롬프트

복사

내 온라인 명함 웹페이지를 card.html 파일 하나로 만들어줘. 이름, 소속, 한 줄 소개, 이메일이 들어가고, 세련된 다크 테마로. 내용은 일단 아무거나 채워놔 — 내가 나중에 알려줄게.

📋 이어서 — 내 정보로

복사

이름은 ○○○, 소속은 ○○팀으로 바꿔줘. 한 줄 소개는 "웨이퍼와 씨름하는 엔지니어" 느낌으로 네가 몇 개 제안해줘.

🚀 더 해보기

🚀 "마우스 올리면 반짝이는 효과 넣어줘"

👁️ 예상 결과

**김공정** · 식각기술팀

"웨이퍼와 씨름하는 엔지니어"

engineer@fab.example

브라우저에 뜨는 다크 테마 카드 한 장

✅ 완료 조건

- ✅ `card.html` 이 브라우저에서 열린다
- ✅ 수정 요청을 2번 이상 해봤다

🚂 막혔다면

💡 "방금 만든 파일 경로 알려줘" / 에러는 그대로 복붙

수정 요청도 기술이다 — 그대로 말하면 됩니다

👁️ 눈에 보이는 대로

- "버튼이 너무 작아, 두 배로"
- "전체적으로 촌스러워, 더 고급스럽게"
- "모바일에서 보면 깨져, 고쳐줘"

🎬 동작을 바꾸고 싶을 때

- "누르면 소리도 나게 해줘"
- "3초는 돌다가 멈추게"
- "이 기능은 빼줘, 복잡해"

"더 예쁘게"도 훌륭한 지시입니다. 디자이너에게, 신입사원에게 말하듯 하면 됩니다.

버튼을 누르면 움직이는 것. 오늘 점심은 이걸로 정합니다. 진짜로.

📄 시작 프롬프트

[복사](#)

점심 메뉴 룰렛을 lunch.html 파일 하나로 만들어줘. 돌림판이 빙글빙글 돌다가 하나를 뽑아주는 애니메이션으로. 메뉴는 김치찌개, 돈까스, 초밥, 쌀국수, 샐러드, 구내식당, 짜장면, 버거 8개. 시작 버튼은 크고 누르고 싶게.

📄 이어서 — 감질나게

[복사](#)

당첨 순간에 폭죽 효과랑 두구두구 하는 긴장감을 추가해줘. 메뉴를 직접 추가/삭제할 수 있는 입력칸도.

🚀 더 해보기

🚀 "같은 메뉴가 3번 연속 나오면 안 되게 해줘"

👁️ 예상 결과



💣 돈까스!

폭죽 + 두구두구 효과
메뉴 추가/삭제 입력칸

✅ 완료 조건

- ✔️ 돌림판이 돌고 결과가 나온다
- ✔️ 오늘 점심이 정해졌다 🍜

45분 전의 나에게 없던 것

2개

동작하는 웹 도구

0줄

직접 작성한 코드

∞

수정 요청 가능 횟수

- 병목은 코딩이 아니라 "뭘 원하는지 말하기"였습니다.
- 다음: 만들다 보면 반드시 막힙니다 — 빠져나오는 법부터 익힙니다.

PART 2 · 0:45-1:25

고장과 빈칸

잘 되는 것만 해서는 늘지 않습니다.

고장난 걸 살리고, 프롬프트를 직접 써 봅니다.



RESCUE



BLANK — 프롬프트를 드리지 않습니다

2

AI가 틀렸을 때 — 복구 3단계



① 에러 복불

빨간 글씨는 통째로 복사해서 "이 에러 고쳐줘"



② 보여주기·묘사

화면이 이상하면 스크린샷 or "버튼이 겹쳐"처럼 말로



③ 국소·되돌리기

"딱 그것만" 고치고, 더 나빠지면 "방금 거 취소"

- 막히는 건 **실패가 아니라 정상**입니다 — 프로 개발자도 하루 종일 에러를 봅니다.
- 여러분이 할 일은 **진단이 아니라 전달**입니다. 증상만 넘기면 원인은 AI가 찾습니다.

복구 문장 사전

—그대로 복붙

😵 이르면 걸립니다

"안 돼요"

"에러 나요"

"이상해요"

→ AI가 뭘 고쳐야 할지 모름

😎 이렇게 전달하세요

"이 에러 메시지 그대로야: [붙여넣기]. 고쳐줘"

"화면이 하얗게만 나와. 콘솔 에러를 확인하고 고쳐줘"

"차트는 나오는데 색이 반대야. 90% 이상이 초록이어야 해"

"방금 수정하고 더 깨졌어. 직전 버전으로 되돌려줘"

💡 브라우저 콘솔 여는 법

💡 **F12** (또는 우클릭 → 검사) → **Console** 탭 → 빨간 줄을 복사

일부러 3군데 고장 낸 수율 차트입니다. 복구 3단계로 하나씩 살려보세요.

📁 buggy_yield.html 받기

열어보면 화면이 백지 — 콘솔부터 보세요

📋 ① 백지 화면 살리기

복사

buggy_yield.html을 열었더니 화면이 하얗게만 나와. 이 파일을 읽고 브라우저 콘솔 에러를 확인해서 고쳐줘. 차트가 뜨게만 먼저.

📋 ② 색·라벨 바로잡기 (묘사로)

복사

이제 차트는 나와. 그런데 ①수율이 90% 이상인 랫이 빨강, 미만이 초록으로 색이 반대야. ②제목이 "두께"인데 사실 수율 데이터야. ③y축에 값 눈금이 없어. 이 세 가지를 고쳐줘.

🚀 더 해보기

🚀 "90% 기준선을 점선으로" / "고장 나기 전후를 비교 설명해줘"

👁️ 예상 결과

✖ 백지 + Console → 고침 → ✔ 차트 정상

수율(%) - 저수율 3랏 빨강



저수율(빨강) LOT-03-05-08이 제대로 보임

✔ 완료 조건

✔ 백지 → 차트가 뜬다 (①)

✔ 색·제목·눈금이 맞다 (②)

✔ 에러 복붙 / 묘사 두 방법을 다 써봤다

실습 데이터 팩

 data.zip 한번에 받기

전부 합성 데이터이고, 실습마다 찾아내야 할 "정답"이 심어져 있습니다. 압축을 풀어 실습 폴더에 두세요.

파일	내용	실습
thickness.csv	막 두께 측정 (5랏 × 5매 × 9포인트)	LAB B-1
wafermap.csv	다이별 테스트 bin (1,257 다이)	부록 C-1
tool.log	장비 로그 24시간 1,000줄	부록 C-2 · E-3
trend.csv	4주차 랏 CD 계측	부록 C-3
spc.csv	60랏 SPC 측정값	LAB P-1
yield.csv	8주차 수율 + bin 카운트	부록 D-1
weekly/ · specs/ · manual.txt	주간 가동 CSV 5개 · 스펙 문서 3개 · 매뉴얼	부록 E-2 · E-5 · B-6
fab_runs.csv · pm_assets.csv · wafermap_A/B.csv	Fab 런 로그 824건 · 설비 자산 시드 · 웨이퍼 A/B	심화 A-1·A-2·A-3
buggy_yield.html · lot_summary.xlsx	일부러 고장 낸 차트 · 엑셀 원자료(2시트)	LAB R-1 · 부록 E-4
boss/ 별도	보스전 웨이퍼 437다이 + 자동 채점기	LAB D-1

data/boss/ 만 data.zip 밖에 있습니다 — 해당 실습 슬라이드에서 바로 받습니다.

데이터가 없으면, 만들어달라고 하면 됩니다



① 합성 데이터

제공 파일 사용, 또는 "비슷한 가짜 데이터 만들어줘"



② 도구 개발

가짜 데이터로 만들고 다듬기



③ 실전 투입

회사에서, 허용된 환경에서, 진짜 데이터로

- 합성 데이터로 만든 도구는 **컬럼 구조만 같으면** 진짜 데이터에도 그대로 동작 — 연습용 test wafer인 셈.
- 직접 생성할 땐 **정답(이상치·drift)을 심어달라고** 하면 도구를 채점할 수 있습니다.

프롬프트를 드리지 않습니다. 아래 목표만 보고 직접 쓰세요 — 오늘의 진짜 연습입니다.

📁 thickness.csv 받기

컬럼: lot, wafer, site, thickness_nm · 목표 100nm

👉 직접 쓰세요 — 이 뼈대만 채우면 됩니다

파일 이름 을(를) 분석해서

결과물 로 만들어줘.

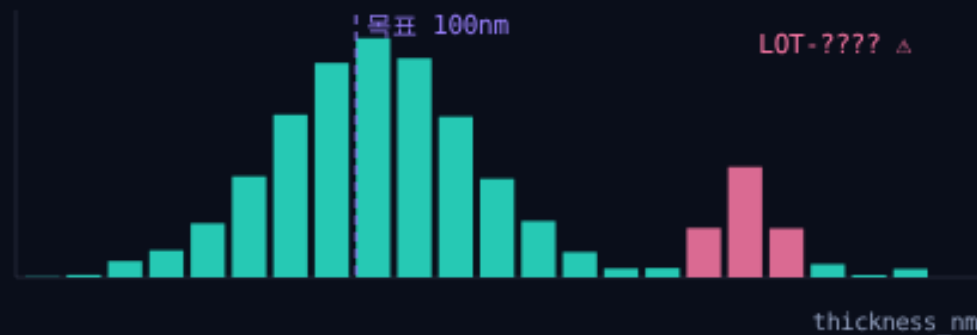
조건은 ① ② ③

▶ 30초 넘게 막혔다면 — 힌트 열기

🚀 더 해보기

🚀 사이트별(중심↔가장자리) 균일도 분석 추가

👁 이 정도가 나오면 성공



✅ 완료 조건

✓ 리포트에 차트가 보인다

✓ 숨어 있던 이상 랫 하나가 잡혔다

✓ 내가 쓴 프롬프트를 한 번 이상 고쳐 봤다

💡 이 실습의 진짜 목표

💡 정답이 숨어있는 데이터 = 도구를 채점할 수 있는 데이터

여기까지 나온 프롬프트 패턴 4장



번호 목록

요구사항을 1,2,3으로 → 그대로 검수 체크리스트



형식 지정

"[실적/이슈/계획] 3섹션, 개조식으로"
— 틀을 정해주기



계획 먼저

"아직 만들지는 마" — 목차·계획 확인 후 제작



스스로 검증

"뭘 바꿨는지 보고해줘", "알려진 값으로
검산해줘"

방금 **직접 쓴 프롬프트**에 이 중 몇 장이 들어 있었나요? 빠진 카드가 다음에 쓸 카드입니다.



체크포인트

잠깐 멈추고, **내 상태만** 점검합니다. 발표도 공유도 없습니다.

확인

지금까지 만든 파일이 어디에 있는지 알고
있나요? 모르면 "**방금 만든 파일 경로 알려
줘**"

정리

안 되는 게 하나라도 있으면 **Q&A에 익명
으로** 남겨 주세요.

강사

공통 질문만 골라 전체에게 한 번에 답합
니다.

🕒 3분

PART 3 · 1:40-2:30

채점과 의심

정답이 있는 과제로 내 도구를 채점하고,
내가 만든 결과를 내가 의심해 봅니다.

 자동 채점 보스전

 셀프 검증





웨이퍼 하나에 정답 5개가 숨어 있습니다. CSV를 눈으로 읽지 말고 **도구를 만들어** 답을 뽑아내세요.
채점기가 즉시 점수를 매깁니다 — **100점이면 손 드세요.**

🏆 채점기 열기

📁 boss_wafermap.csv

컬럼: die_x, die_y, bin (1=양품)

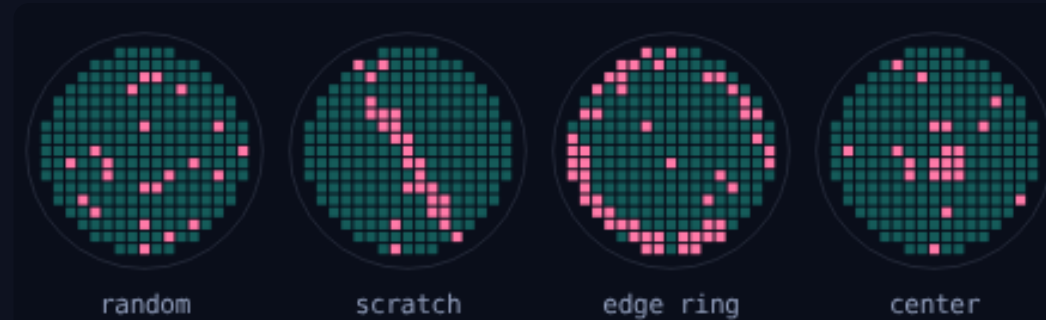
🎯 채점 문항

#	물어볼 것	배점
Q1	총 다이 개수	20
Q2	수율 % (bin 1 = 양품)	20
Q3	최다 불량 bin 번호	20
Q4	불량의 공간 패턴	20
Q5	불량을 최악 사분면	20
합계		100

💡 이기는 순서

- 💡 Q1(개수)부터 — 도구가 파일을 **제대로 읽는지** 확인하는 관문
- 💡 Q4·Q5는 **웨이퍼 맵을 그려 놓고** 보면 3초

👁️ Q4 패턴 후보 — 이 넷 중 하나



맵을 화면에 그려 놓으면 Q4·Q5는 눈으로 끝납니다 (이 그림은 예시일 뿐, 답이 아닙니다)

✅ 완료 조건

- ✅ 채점기에서 **60점 이상**
- ✅ 답을 **손으로 세지 않았다** — 도구가 뽑았다

🚀 만점자 추가 미션

🚀 "다른 CSV를 넣어도 같은 5개 답이 나오게" 일반화

믿지 말고 확인한다 — 검증 습관 3가지



정답 심기

이상치·drift가 심어진 데이터로 도구를 채점한다



개수 세기

"ERROR 8건" — 리포트 숫자를 직접 세본다



아는 값 넣기

760 Torr = 101325 Pa처럼 답을 아는 입력으로
검산

AI의 결과는 **계측 장비의 데이터처럼** 다루세요 — 캘리브레이션 없이 믿지 않기.

큰 것은 쪼개서 시킨다



데이터 먼저
CSV 확인



차트 하나
동작 확인 후



차트 추가
하나씩 늘리기



조립·꾸미기
배치와 인터랙션



한 방에

"수율 분석 대시보드 만들어줘" → 복불복, 고치기도 어려움



단계별

"일단 트렌드 차트부터. 확인하고 다음 거 시킬게" → 매 단계 검증 가능

만드는 것은 절반입니다. 나머지 절반은 내가 만든 결과를 내가 의심하는 것.

[📁 spc.csv 받기](#)

50개 측정 · 룰 위반이 2건 심어져 있음

📋 ① 만들기

[복사](#)

spc.csv로 SPC 관리도(x-bar chart)를 spc.html로 만들어줘. 평균선(CL)과 $\pm 3\sigma$ 관리선(UCL/LCL)을 그리고, Western Electric 룰 1(관리선 이탈)과 룰 2(9점 연속 한쪽)를 위반한 점을 빨강게 표시해줘. 위반 목록도 표로 같이.

📋 ② AI에게 검사까지 시키기

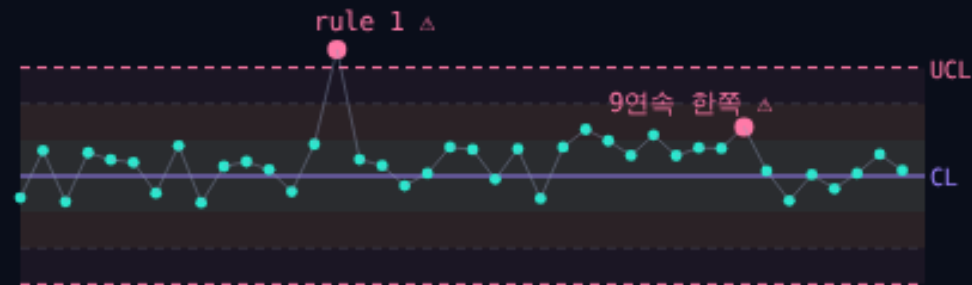
[복사](#)

방금 만든 결과를 네가 직접 검사해서 표로 보여줘. UCL/LCL 값을 원본 데이터로 다시 계산하고, 위반으로 표시한 점이 정말 위반인지 근거(몇 번째 점, 어떤 룰)를 하나씩 적어줘. 틀린 게 있으면 고쳐줘.

🚀 더 해보기

🚀 "룰 3~4까지 추가하고, 각 룰을 한 줄로 설명해줘"

👁️ 예상 결과



🔍 스스로에게 던지는 질문 3개

- ① UCL 값이 얼마인가? **손으로도 맞나?**
- ② 위반 2건이라는데, **그 줄을 원본에서** 찾을 수 있나?
- ③ 9점 연속을 **어떻게** 썼는지 설명할 수 있나?

✅ 완료 조건

- ✓ 관리도가 그려지고 위반 **2건**이 잡혔다
- ✓ 숫자 하나를 **원본과 대조**했다
- ✓ ②번 프롬프트로 **AI에게 검사를 시켜** 봤다

도메인 지식 × AI = 여러분의 무기

여러분이 낸 것

- WE rules라는 개념, Rayleigh라는 식
- 뭐가 정상이고 뭐가 이상한지의 감각

AI가 낸 것

- 구현 코드, 차트·UI 전부
- 지치지 않는 반복 수정

같은 AI라도 도메인 전문가가 시키면 결과가 다릅니다.

개발자가 되는 게 아니라, 더 강한 엔지니어가 되는 겁니다.

PART 4 · 2:30-3:00

이제 내 업무로

오늘 배운 걸 전부 들고, **내가 매주 반복하는 그 일**로 옵니다.

오늘의 마지막 실습이자, 유일하게 주제가 각자 다른 실습입니다.



내 업무 도구 하나



다음에도 쓰게 굳히기



자동화 대상 찾기 — 셀프 문진표

이 중 하나라도 "예"라면 후보

- 매주/매일 **같은 순서로** 클릭하는 일이 있다
- 파일 열어 **복사-붙여넣기-저장**을 반복한다
- "이건 인턴 시키면 되는데" 싶은 일이 있다

자동화 설계 공식 — 한 문장

"__(입력)__을 받아서
__(규칙)__대로 처리하고
__(출력)__을 만들어줘"

못 채우겠으면 AI에게 같이 채워달라고 하면 됩니다.

 잠깐: 지금 **내 업무에서 후보 하나**를 적어두세요 — 마지막 파트에서 그걸 만듭니다.

마지막 실습에서 만들 것을 정합니다

🕒 5분

1 내가 **매주 반복**하는 귀찮은 일 하나는

예: 매주 월요일 장비별 알람 로그를 훑어서 보고서에 옮겨 적기

2 그 일의 **입력**은 (무엇을 받아서)

예: 장비에서 내려받은 CSV 5개

3 원하는 **출력**은 (무엇이 나오면 끝)

예: 알람 상위 5개와 시간대 차트가 있는 HTML 한 장

3분 안에 안 떠오르면 — **다음 장 아이디어 카드에서 하나 집으세요.** 고민하는 시간보다 만드는 시간이 훨씬 남는 장사입니다.

주제가 안 떠오르면 — 여기서 고르세요

🕒 30초 안에

 교대 인수인계 노트

쓰면 자동 정리·검색되는 시프트 로그

 장비 예약 보드

측정기 예약 현황 한눈에 + 중복 방지

 레시피 비교기

파라미터 파일 2개의 차이만 하이라이트

 실험 노트 정리기

날짜·조건·결과를 표준 양식으로 축적

 DOE 조건표 생성기

인자·수준 입력 → 실험 조건 매트릭스

 일일 지표 대시보드

내가 매일 보는 숫자 4개를 한 화면에

 PM 일정 알리미

주기 입력 → 다가오는 정비 일정 캘린더

 회의용 차트 공장

CSV 던지면 보고서용 차트 3종 자동 생성

 파일명 검색기

수천 개 측정 파일에서 조건으로 찾기

 우리 팀 계산기

팀에서 자주 쓰는 계산식 모음 웹앱

 체크리스트 앱

셋업/릴리즈 절차를 클릭형 체크리스트로

 신입 온보딩 퀴즈

우리 공정 기초 지식 퀴즈 게임

오늘의 **마지막 실습**. 주제는 각자 다릅니다 — 앞 실습들이 전부 이걸 위한 준비였습니다.

📄 만능 첫 프롬프트 — ○○만 내 걸로

[복사](#)

나는 반도체 ○○ 업무를 하는 엔지니어야. 매주 ○○하는 일이 번거로워서 자동화하고 싶어.
입력은 ○○이고, 원하는 출력은 ○○이야.

25분 안에 만들 수 있는 v1 계획을 먼저 3줄로 제안해줘. 내가 좋다고 하면 그때 만들기 시작해줘.

연습용 가짜 데이터도 같이 만들어줘 — 실데이터는 안 쓸 거야.

💡 25분을 지키는 법

💡 계획이 4줄을 넘으면 **"절반으로 줄여줘"**

💡 기능은 **1개만** — 욕심이 제일 큰 적입니다

💡 막히면 오늘 배운 **복구 문장 사전**으로

✅ 완료 조건 — 이 셋이면 통과

✓ 가짜 데이터로 한 번은 **끝까지 돌아간다**

✓ 결과 숫자 하나를 **원본과 대조했다**

✓ 내일 이걸 **어디에 쓸지** 말할 수 있다

⚠️ 이 25분에 하지 말 것

× 실데이터 열기 — 모양만 흉내 낸 가짜 데이터로

× "우리 팀 전체 시스템"급 대작 (그건 다음 주부터)

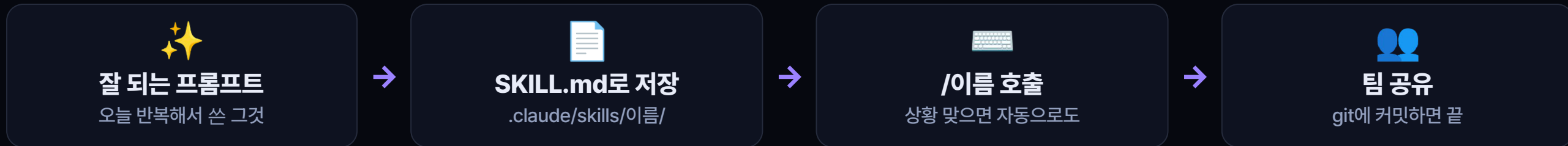
× 완성 욕심 — **동작하는 반쪽**이 계획서 100장보다 낫습니다

🚀 빨리 끝났다면

🚀 부록 실습에서 **내 주제와 가까운 것** 하나를 골라 이식

잘 먹힌 프롬프트, 나만의 스킬로 저장

오늘 여러 번 재사용한 그 프롬프트, 매번 다시 설명하지 마세요. 스킬(Skill)로 저장하면 다음부터 `/이름` 한 줄. 팀 공유도 파일 하나. 프롬프트 자동화의 끝판입니다.



- 스킬은 `SKILL.md` 파일 하나입니다 — 맨 위에 `description` (언제 쓰는지), 그 아래에 **지시문**.
- **description이 방아쇠**: 그 상황이 오면 Claude가 알아서 이 스킬을 꺼내 씁니다. `/이름` 으로 직접 부를 수도 있고요.
- 위치가 범위를 정합니다 — `.claude/skills/` (이 프로젝트만) vs `~/.claude/skills/` (내 모든 프로젝트). 다른 도구 필요 없이 파일만 만들면 됩니다.

오늘 잘 먹힌 프롬프트를 **파일로** 남깁니다. 다음 주의 나는 오늘의 나를 기억하지 못합니다.

📄 그대로 시키세요 — 이 한 줄로 끝

복사

오늘 이 도구를 만들면서 우리가 주고받은 지시 중에서 잘 통했던 것들을 정리해줘.
다음에 비슷한 걸 또 만들 때 이 파일 하나만 열면 되도록 SKILL.md 로 저장해줘.

- 이 도구가 뭘 하는지 3줄
- 다시 만들 때 그대로 쓸 프롬프트 순서
- 내가 실제로 겪은 함정과 해결책

💡 굳히는 방법은 하나가 아닙니다

💡 Claude Code라면 `.claude/skills/` 에 스킬로

💡 아니면 `PROMPTS.md` 메모 한 장이면 충분합니다

✅ 완료 조건

- ✅ 파일 하나가 남았다 (SKILL.md 또는 메모)
- ✅ 거기에 내가 쓴 프롬프트가 들어 있다

👁️ 예상 결과

SKILL.md

주간 알람 리포트 만들기

1. "CSV 5개를 읽어서 장비별로 묶어줘"
2. "시간대 분포 차트를 붙여줘"
3. ⚠ 한글 깨지면 → "UTF-8로 다시 저장"

마무리 공유

희망자만·2분

1

뭘 만들었나

화면 시연 30초 — 말보다 보여주기. **안 굴러가도
켜 놓으세요.**

2

어떤 프롬프트가 잘 먹혔나

오늘의 진짜 지식 공유는 이것

3

어디서 막혔나

막힌 얘기가 남에게 제일 유용합니다

부담 없이. 보여주고 싶은 사람만 하면 됩니다. 아무도 없으면 강사가 오늘 나온 결과물 중 몇 개를 소개하고 끝냅니다.

만든 걸 팀에 넘기는 4가지 방법



파일 그대로

HTML 파일 하나면 끝. 메일·메신저로 보내면 받는 사람은 **더블클릭**만.



공유 드라이브

사내 공유폴더/그룹웨어에 올려두고 링크 공유. 버전은 파일명에 날짜.



사내 웹 호스팅

GitHub Pages·사내 정적 호스팅에 올리면 **URL 하나**로 상시 접속.



QR·바로가기

회의실에서 QR로 띄우기, 바탕화면 바로가기로 매일 쓰기.



배포도 시키면 됩니다

[복사](#)

이 도구를 팀원들이 쓰기 쉽게 배포하려고 해. 파일 하나로 합치고(외부 라이브러리도 안에 포함), 열었을 때 사용법 안내가 뜨게 해줘. 그리고 이 페이지로 가는 QR 코드도 하나 만들어줘.

실데이터로 넘어가기 전 — 체크리스트

오늘은 전부 합성 데이터였습니다. 회사에서 **진짜 데이터**에 쓰기 전에 이것만은.

✓ 도입 전 확인

- ✓ 사내 보안 정책이 **허용한 도구·환경**인가
- ✓ 합성 데이터로 **먼저 검증**했는가 (정답 심어 채점)
- ✓ 데이터가 어디로 나가는지 아는가 (로컬 처리 우선)
- ✓ 중요한 판단에 쓴다면 **검산 루틴**이 있는가



그래도 넣지 말 것

공정 레시피·실측정치·고객정보·미공개 제품명. 애매하면 **보안 담당자에게 먼저**. 도구는 합성 데이터로 만들고, 실데이터는 허용된 환경에서만 흘려보냅니다.

오늘의 프롬프트 패턴 — 지시 편

패턴	이렇게 말한다
 번호 목록	"요구사항: 1..., 2..., 3..." — 그대로 검수 체크리스트가 된다
 형식 지정	"[실적/이슈/계획] 3섹션, 개조식으로" — 출력 틀을 먼저 정한다
 계획 먼저	"아직 만들지 마. 계획/목차/변경표부터 보여줘"
 쪼개기	"일단 차트 하나만. 확인하고 다음 걸 시킬게"
 맥락 주기	"나는 ○○하는 엔지니어야. 이걸 ○○에 쓸 거야"
 지식 주입	공식·룰·용어를 프롬프트에 직접 써준다 ($R = k1 \cdot \lambda / NA$)

오늘의 프롬프트 패턴 — 검증·안전 편

패턴	이렇게 말한다
 정답 심기	합성 데이터에 이상치를 심고, 도구가 찾아내는지 채점한다
 블라인드	"정답은 answer.txt에 저장해. 분석할 땐 읽지 마"
 자기 보고	"뭘 바꿨는지/몇 개 처리했는지 보고해줘"
 검산 시키기	"알려진 값으로 검증해서 표로 보여줘"
 출처 강제	"문서 근거를 인용하고, 없으면 없다고 말해"
 안전 작업	"원본은 두고 사본으로", "계획표 확인 후 실행"
 에러 복붙	에러 메시지는 해석 말고 통째로 붙여넣기 + "고쳐줘"

내일부터 — 습관으로 만들기



1주차

오늘 만든 것 중 하나를 **진짜 업무에 투입**. 매일 한 번은 AI에게 뭐든 시켜보기.



1개월

주 1개씩 **반복 업무 은퇴**시키기. 잘 먹힌 프롬프트를 팀 위키에 축적.



3개월

팀 공용 도구 하나 운영. 옆 동료에게 **오늘 배운 걸 전파** — 가르치면 확실히 는다.

⚠ 잊지 마세요: **사내 보안 정책 확인** → 허용된 도구·환경에서 → 실데이터는 합성 데이터 검증 후에. 그리고 중요한 결정에 쓰는 도구는 반드시 검산 루틴과 함께.

더 보기

- 📖 개념 다시 보기: [바이브 코딩이란 무엇인가? \(본 세미나 슬라이드\)](#).
- 🧠 한 단계 깊게: [컨텍스트 — AI 에이전트의 작업 책상](#) (긴 작업이 산으로 갈 때 읽어보세요)
- 📋 이 슬라이드: 모든 실습 프롬프트에 복사 버튼이 있습니다 — 두고두고 참고서로 쓰세요
- 📦 실습 데이터 팩: [data.zip](#) — 정답이 심어진 합성 데이터 8종, 집에서 복습용

20+

오늘 실행한 프롬프트

10+

만들어낸 도구와 리포트

1개

내일 실제로 쓸 내 도구

THANK YOU

오늘 아침의 여러분은 이걸 못 만들었습니다

지금의 여러분은 만듭니다. 그 차이가 전부입니다.

내일 아침, 반복 업무 하나를 골라 은퇴시키세요.



Q&A



피드백 환영

APPENDIX · 부록

혼자 할 실습 21개

본편에서 다루지 않은 실습 21개입니다. 잘라낸 게 아니라 남겨 둔 것입니다.
풀데이 코스에서는 이 중 골라서 진행합니다.



전부 프롬프트 포함



집에서 각자



부록 실습, 이렇게 쓰세요

본편 실습은 형식을 계속 바꿔가며 루프를 몸에 익히는 것이 목적이었습니다. 부록은 반대입니다 — 필요할 때 골라 쓰는 레시피 모음입니다. 순서대로 다 할 필요가 전혀 없습니다.



내 업무와 가까운 것부터

21개를 순서대로 하지 마세요. 지금 내 일에 닿는 것 **1~2개**면 충분합니다.



프롬프트는 가리고

제목과 완료 조건만 보고 **먼저 직접 써 보기** — 그
래야 실력이 늡니다.



내 도구에 이식

새 파일을 만들기보다 **오늘 만든 도구에 기능으로**
붙이는 편이 낫습니다.

가장 좋은 다음 한 걸음: 오늘 만든 도구를 내일 업무에서 한 번 켜 보는 것. 부록 21개보다 그게 낫습니다.

미니 프로젝트 규칙

✓ 이렇게

- **작게:** 45분에 v1이 나올 크기로 (기능 1~2개)
- **합성 데이터로:** 실데이터 모양만 흉내
- **계획 먼저:** "어떻게 만들지 계획부터 보여줘"
- **단계별로:** 하나 되면 다음 것

⚠ 이것만 피하기

- 사내 기밀·실데이터 사용 ✗
- "우리 팀 전체 시스템"급大作 ✗ (그건 다음 주부터)
- 완성 욕심 — **동작하는 반쪽**이 계획서 100장보다 낫습니다

📋 시작이 막막할 때 — 만능 첫 프롬프트

복사

나는 반도체 ○○ 업무를 하는 엔지니어야. 매주 ○○하는 일이 번거로워서 자동화하고 싶어. 입력은 ○○이고, 원하는 출력은 ○○이야. 45분 안에 만들 수 있는 v1 계획을 먼저 제안해줘. 연습용 가짜 데이터도 같이 만들어줘.

처음으로 업무 냄새가 나는 도구 — 매번 검색하던 단위 변환을 내 것으로.

📄 시작 프롬프트

복사

반도체 엔지니어용 단위 변환기를 unit.html 파일 하나로 만들어줘. nm ↔ Å ↔ μm ↔ mil 길이, Torr ↔ Pa ↔ mbar 압력, °C ↔ K 온도 세 가지 탭으로. 숫자를 입력하면 실시간으로 전부 같이 변환돼서 보이게.

📄 이어서 — 검증까지 시키기

복사

1 Å = 0.1 nm, 760 Torr = 101325 Pa 같은 알려진 값으로 변환이 정확한지 표로 검증해서 보여줘.

🚀 더 해보기

🚀 sccm ↔ slm 유량, eV ↔ nm 광자 에너지 탭 추가

👁️ 예상 결과

길이 · 압력 · 온도

1 Torr	= 133.322 Pa	= 1.333 mbar
760 Torr	= 101325 Pa	= 1013.25 mbar

입력하면 모든 단위가 동시에 갱신

✅ 완료 조건

✓ 세 종류 변환이 모두 동작

✓ 아는 값으로 검산해봤다 (760 Torr)

요구사항 여러 개를 처음부터 번호로 정리해서 시켜봅니다.

📄 시작 프롬프트

[복사](#)

회의 타이머를 timer.html 파일 하나로 만들어줘. 요구사항:

1. 발표 시간을 분 단위로 설정 (기본 5분)
2. 남은 시간이 화면 가득 크게 보이게
3. 1분 남으면 노란색, 다 되면 빨간색 + 깜빡임
4. 시작/일시정지/리셋 버튼

회의실 프로젝터에 띄워놓고 쓸 거야.

🚀 더 해보기

🚀 "발표자 명단을 넣으면 순서대로 돌아가게"

👁️ 예상 결과

00:58

☐ 5분+ → ☒ 1분 미만 → ☐ 깜빡임

✅ 완료 조건

- ✓ 타이머가 돌고 색이 바뀐다
- ✓ 요구사항 4개를 **하나씩 체크**했다

💡 포인트

💡 번호 요구사항 = 그대로 **검수 체크리스트**. 빠지면 "3번이 안 됐어"

빨리 끝난 사람용 — 내 최고 기록을 깨보는 것.

📄 시작 프롬프트

[복사](#)

반응속도 측정 게임을 reaction.html로 만들어줘. 화면이 빨간색이었다가 랜덤한 순간 초록색으로 바뀌면 최대한 빨리 클릭하는 게임. 반응 시간을 ms로 보여주고, 5회 평균과 최고 기록을 기록해줘. 너무 일찍 누르면 "성급했습니다!" 하고 놀려줘.

🚀 더 해보기

🚀 "등급을 매겨줘 — 200ms 미만은 'FAB 알람 대응 S급'"

👁️ 예상 결과

⚡ **236** ms

5회 평균 251 ms · 최고 198 ms — FAB 알람 대응 A급

✅ 완료 조건

- ✓ 내 평균 반응속도를 알아냈다
- ✓ 최고 기록이 저장되고 갱신된다

중구난방 회의 메모 → 정리된 회의록.

📄 시작 프롬프트 (샘플 메모 포함)

복사

아래 회의 메모를 정리해서 minutes.md 파일로 만들어줘. ①3줄 요약 ②결정사항 ③액션 아이템(담당자/기한 표) 순서로.

--- 메모 ---

설비팀 김프로: 3번 챔버 파티클 이슈 지난주부터 재발, 원인 미상. 정비 일정 협의 필요

공정팀: 신규 레시피 평가 지연중, 측정 장비 예약이 밀림. 다음주 화요일까지 1차 결과 공유하기로 함

품질팀 이프로: 월간 감사 준비, 각 팀 문서 목록 금요일까지 제출 요청

팀장: 파티클 이슈는 설비팀+공정팀 합동으로 이번주 목요일까지 1차 원인 분석. 레시피 평가는 우선순위 상향

🚀 더 해보기

🚀 "팀 공지용 3줄 버전도 따로" / 우리 팀 회의록 양식 틀에 맞추기

👁️ 예상 결과

📄 minutes.md — 요약 3줄 + 결정 2건 +

액션	담당	기한
파티클 1차 원인 분석	설비+공정	목요일
감사 문서 목록 제출	각 팀	금요일

✅ 완료 조건

✓ 요약/결정/액션이 구분돼 나온다

✓ 액션에 담당자·기한이 표로 정리됐다

일주일치 낙서 같은 메모 → 보고서 문체.

📄 시작 프롬프트

[복사](#)

아래 이번 주 업무 메모로 주간보고 초안을 작성해줘. 형식: [금주 실적 / 이슈 및 리스크 / 차주 계획] 3개 섹션, 개조식 문어체("~함", "~완료")로.

- 화: 신규 장비 셋업 지원, 캘리브레이션까지 완료
- 수: 파티클 이슈 미팅. 원인 후보 2개로 좁힘 (밸브 실링 or 배기 라인)
- 목: 분석 데이터 정리하다 측정기 예약 못 잡아서 반나절 날림
- 금: 보고서 초안 작성 예정. 다음주엔 재현 실험 하려고 함

📄 이어서 — 톤 조절

[복사](#)

"반나절 날림"처럼 부정적으로 보이는 부분은 리스크와 대응 계획 형태로 다듬어줘. 임원 보고에 들어가도 어색하지 않게.

👁️ 예상 결과

■ 금주 실적

· 신규 장비 셋업 및 캘리브레이션 완료

■ 이슈 및 리스크

· 계측 리소스 경합으로 분석 일정 지연 → 예약 선점으로 대응

■ 차주 계획

· 파티클 재현 실험 수행 예정

✅ 완료 조건

- ✓ 3개 섹션 개조식 보고서가 나왔다
- ✓ 톤 조절 요청까지 해봤다

상황만 한국어로 설명하면 정중한 영문 메일이 나옵니다.

📄 시작 프롬프트

[복사](#)

장비 벤더 엔지니어에게 보낼 영문 메일을 써줘. 상황: 챔버 압력 센서가 간헐적으로 이상값을 보여서 지난주에 문의했는데 아직 답이 없음. 정중하지만, 다음 주 PM 일정 전에는 꼭 답을 받아야 한다는 긴급함이 전달되게. 제목도 같이. 메일 아래에 한국어 번역도 붙여줘.

📄 이어서 — 뉘앙스 조절

[복사](#)

더 부드러운 버전과 더 단호한 버전 두 가지로 만들고, 어떤 상황에 어떤 걸 쓰면 좋을지도 알려줘.

👁️ 예상 결과

Subject	Follow-up: Intermittent pressure sensor readings — response needed before PM (July 20)
Body	Dear Mr. Tanaka, I'm following up on my inquiry from last week...
버전	🌀 부드럽게 / ⚡ 단호하게 + 사용 시점 가이드

✅ 완료 조건

- ✓ 제목 + 본문 + 한국어 번역
- ✓ 두 가지 톤 버전을 비교해봤다

🚀 더 해보기

🚀 받은 영문 메일에 대한 회신 / 일본어 버전

표기가 제각각인 목록 → 정돈된 표 + CSV.

📄 시작 프롬프트 (지저분한 데이터 포함)

[복사](#)

아래 장비 목록을 정리해서 표로 만들고 equipment.csv로 저장해줘. 컬럼: 장비ID / 공정 / 상태 / 비고. 표기가 제각각인 건 통일하고, 뭘 어떻게 통일했는지도 알려줘.

ETCH-01 식각 가동중

etch02 / Etch / PM예정(금요일)

CVD 3번장비 - 증착 - idle

cvd-04, 증착, 가동 중, 신규셋업

PHOTO_7 노광 다운 (파츠 대기)

photo8 노광 가동중

👁️ 예상 결과

장비ID	공정	상태	비고
ETCH-01	식각	가동	
ETCH-02	식각	PM 예정	금요일
PHOTO-07	노광	다운	파츠 대기

+ "etch02→ETCH-02로 통일" 변경 내역 보고

✅ 완료 조건

✓ equipment.csv 가 엑셀에서 열린다

✓ "뭘 통일했는지" 설명을 검증했다

💡 포인트

💡 정리 규칙을 스스로 보고하게 하면 검증이 쉬움 — 행이 수백 개여도 동일

주제만 주면 목차 → 슬라이드까지. 지금 이 자료도 이렇게 만들었습니다.

📋 프롬프트 ① — 목차 먼저

복사

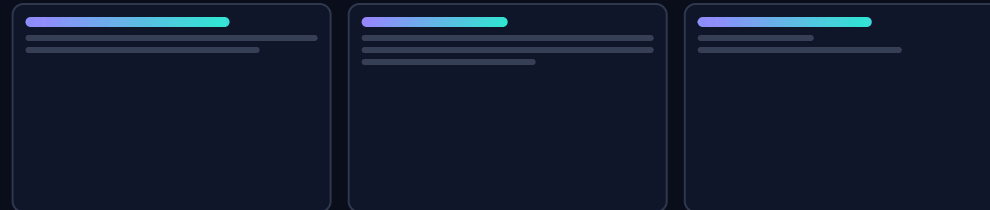
"신입 엔지니어를 위한 클린룸 기본 수칙" 5분 발표 자료를 만들려고 해. 먼저 슬라이드 6장 분량의 목차와 각 장의 핵심 메시지를 제안해줘. 아직 만들지는 마.

📋 프롬프트 ② — 확인 후 제작

복사

좋아. 3번 슬라이드는 복장 착용 순서를 단계별 그림처럼 바꿔줘. 그 목차대로 slides.html 파일 하나로 만들어줘. 화살표 키로 넘길 수 있게, 깔끔한 다크 테마로.

👁️ 예상 결과



화살표로 넘기는 6장짜리 미니 발표자료

✅ 완료 조건

- ✓ 목차 단계에서 수정 지시를 했다
- ✓ 슬라이드가 브라우저에서 넘어간다

💡 포인트

💡 "아직 만들지는 마" — 목차에서 고치는 게 10배 싸입니다

긴 문서를 읽는 대신 문서에게 질문합니다.

📁 제공 데이터 `manual.txt` 받기

가상 장비 유지보수 매뉴얼 (실습 폴더에 저장)

📋 시작 프롬프트

복사

`manual.txt`를 읽고 답해줘: ①월간 점검 항목만 표로 정리 ②"경고"가 붙은 항목 전부 나열 ③신입에게 알려줄 핵심 3가지. 반드시 문서에 있는 내용만 근거로 하고, 문서에 없으면 없다고 말해줘.

🚀 더 해보기

🚀 "PG-2가 무응답이면 어떻게 해?" 같은 고장 대응 질문 던지기

👁️ 예상 결과

"경고" 붙은 항목 전부 나열해줘

경고는 총 4건입니다: ①뷰포트 개방 전 대기압 확인 ②그리스 작업 전 LOTO ... — `manual.txt` 2·4·5장

✅ 완료 조건

- ✓ 문서 근거 답변을 받았다
- ✓ 답이 진짜 문서에 있는지 **한 개는 직접 확인**

💡 포인트

💡 "문서에 없으면 없다고 해" — 아는 척 방지 장치

다이별 bin 데이터를 웨이퍼 모양 그대로 색칠해서 봅니다. 오늘의 하이라이트.

[📁 wafermap.csv 받기](#)

컬럼: die_x, die_y, bin (1=양품)

📋 뷰어 제작 프롬프트

[복사](#)

wafermap.csv를 읽어서 웨이퍼 맵으로 보여주는 wafermap.html을 만들어줘.

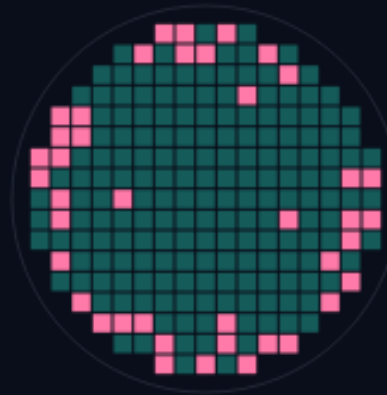
요구사항:

1. 다이를 격자로 그리고 bin별 색 구분 (bin1은 차분하게, 불량은 눈에 띄게)
 2. 마우스를 올리면 좌표와 bin 표시
 3. 우측에 bin별 개수와 수율(%) 요약
 4. bin 번호를 클릭하면 해당 bin만 하이라이트
- 불량이 어디에 몰려 있는지 패턴도 한 줄로 말해줘.

🚀 더 해보기

🚀 여러 웨이퍼 썸네일 갤러리 모드 / notch 방향 표시

👁️ 예상 결과



수율 91.8%

■ bin1 1156

■ bin2 38

■ bin3 27

■ bin4 19

edge ring 패턴

✅ 완료 조건

- ✓ 원형 웨이퍼 맵이 그려진다
- ✓ 불량 패턴이 눈에 보인다 — 무슨 패턴?

🚚 막혔다면

- 💡 "브라우저 콘솔 에러를 확인하고 고쳐줘"

1,000줄 기계 로그에서 필요한 것만 뽑습니다. 사람이 훑으면 20분, 시키면 3초.

[📁 tool.log 받기](#)

24시간 · 1,000줄 · [시각] [레벨] [모듈] 메시지

📋 분석 프롬프트

[복사](#)

tool.log를 분석해서 log_report.html로 만들어줘.

1. ERROR/WARN만 추출해 시간순 표로
2. 메시지 유형별 발생 횟수 집계
3. 시간대별 발생 분포 차트
4. 반복되는 패턴이 있으면 짚어줘

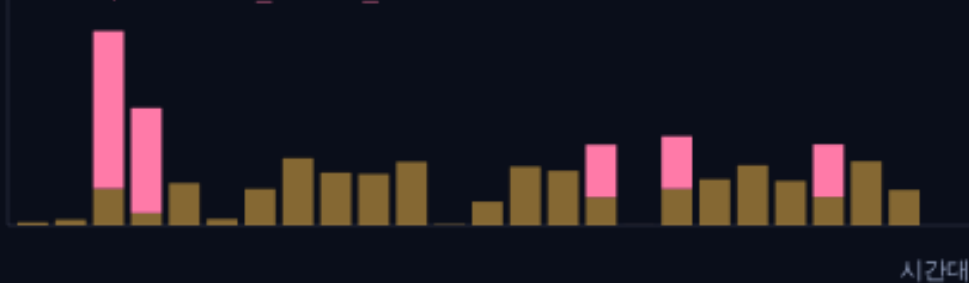
🚀 더 해보기

🚀 "ERROR 전후 5줄씩 문맥도 같이 보여줘"

👁️ 예상 결과

ERROR 8건 · WARN 30건

02-04시 | PRESSURE_SENSOR_TIMEOUT ×3 ▲



✅ 완료 조건

- ✓ ERROR 개수가 정확하다 — 몇 건이었나요?
- ✓ 새벽 반복 패턴을 AI가 짚어냈다

💡 포인트

- 💡 내 장비 로그 형식이 달라도 샘플 몇 줄만 보여주면 됨

부록 C-3 랫 트렌드 차트

🕒 15분



시간에 따른 공정 파라미터 추이. 어딘가에서 drift가 시작됩니다.

📁 trend.csv 받기

4주차 · 하루 6랏 · CD 목표 45nm

📋 차트 프롬프트

복사

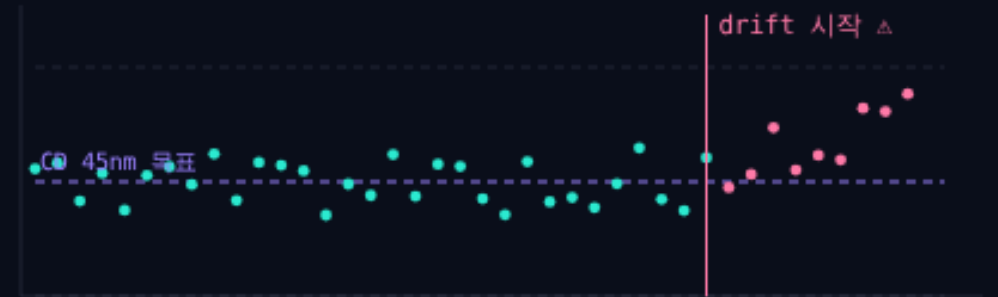
trend.csv로 trend.html 트렌드 차트를 만들어줘.

1. 랫별 CD 값 점 + 일별 평균 선
2. 목표 45nm와 관리선($\pm 1.5\text{nm}$) 표시
3. 구간을 드래그하면 그 구간 평균/표준편차 표시
4. drift가 시작된 지점을 네가 찾아서 표시해줘

🚀 더 해보기

🚀 이동평균선 / "drift 감지를 알림 문구로 요약"

👁️ 예상 결과



✅ 완료 조건

✓ 트렌드에 drift가 눈에 보인다

✓ AI가 찾은 시점이 그럴듯한지 눈으로 검증

"쫄개서 시킨다"를 실전으로 — 차트 3개짜리 대시보드를 단계별로 쌓습니다.

📁 yield.csv 받기

8주 · 제품 A/B · bin2~8 카운트

📋 프롬프트 ② → ③ → ④ — 반드시 하나씩!

복사

② yield.csv로 dashboard.html을 시작하자. 먼저 주차별 수율 트렌드 차트만 (product별 색 구분). 다른 건 아직 만들지 마.

③ 좋아. 이제 그 아래에 불량 bin 파레토 차트를 추가해줘. 기간을 선택하면 그 기간 기준으로 갱신되게.

④ 마지막으로 상단에 KPI 카드(이번주 수율, 전주 대비, 최다 불량 bin)를 얹고 배치를 정리해줘.

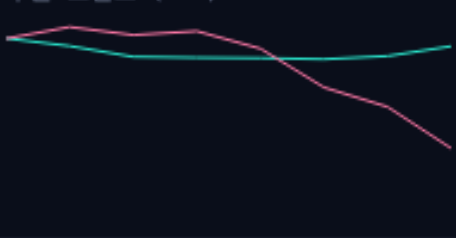
🚀 더 해보기

🚀 "경영진 요약 한 문단을 자동 생성해서 맨 위에"

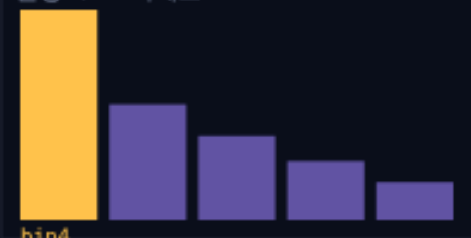
👁️ 예상 결과

이번주 수율
89.1%전주 대비
-1.8%p최다 불량
bin4

수율 트렌드 (B Δ)



불량 bin 파레토



✅ 완료 조건

✓ ②→③→④ 단계마다 열어서 확인했다

✓ 어떤 제품/bin이 문제인지 대시보드에서 읽힌다

🚚 막혔다면

💡 ③에서 ④가 망가짐 → "트렌드 차트가 깨졌어, 둘 다 되게 고쳐줘"

엑셀 구석에 있던 계산식을 팀이 같이 쓰는 도구로. 식은 여러분이 제공합니다.

📄 시작 프롬프트

복사

공정 계산기 모음을 calc.html 파일 하나로 만들어줘. 탭 3개:

1. 포토 해상도: Rayleigh 식 ($R = k1 \cdot \lambda / NA$, $DOF = k2 \cdot \lambda / NA^2$). λ 는 193/248/365nm 선택, $k1 \cdot NA$ 는 입력
 2. 면저항: 4-point probe ($Rs = 4.532 \times V/I$), 두께 입력 시 비저항까지
 3. 식각 시간: 목표 깊이, 식각 속도, 오버에치(%) → 필요 시간
- 각 탭에 식과 변수 설명도 같이.

🚀 더 해보기

🚀 내 업무의 계산식을 4번째 탭으로 (증착·임플란트·CMP...)

👁️ 예상 결과

해상도 · 면저항 · 식각 시간

$\lambda = 193 \text{ nm}$ $NA = 1.35$ $k1 = 0.30$

→ $R = 42.9 \text{ nm}$ · $DOF = k2 \cdot \lambda / NA^2$

값을 바꾸면 결과 즉시 갱신 + 식 표시

✅ 완료 조건

✓ 세 계산기 동작

✓ 아는 값으로 계산 — "교과서 예시로 검증해서 표로 보여줘"

💡 포인트

💡 도메인 공식은 **프롬프트에 직접** — 정확도 급상승

결함 패턴을 만들고, 구분하는 로직까지 — 퀴즈 게임으로.

📄 프롬프트 ① — 시뮬레이터

복사

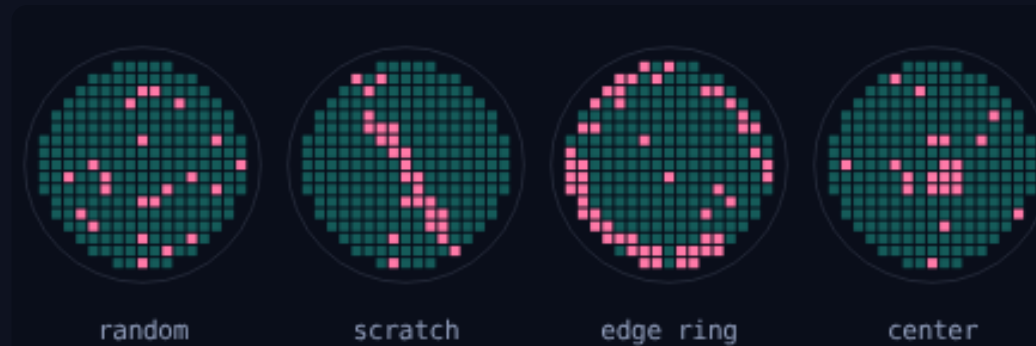
웨이퍼 결함 패턴 시뮬레이터를 defect.html로 만들어줘. 버튼을 누르면 random / scratch(직선형) / edge ring / center cluster 4가지 중 하나로 결함 맵을 생성해서 보여주는 것. 패턴 이름은 숨겨놔.

📄 프롬프트 ② — 자동 분류 퀴즈

복사

"판별" 버튼을 추가해줘. 결함 분포를 분석해 4가지 중 원지 추정하고 근거 지표(가장자리 비율, 직선성, 중심 밀집도)와 함께 표시. 내가 먼저 눈으로 맞히고 답을 확인하는 퀴즈 모드로.

👁️ 예상 결과



✅ 완료 조건

- ✓ 4가지 패턴이 그럴듯하게 생성된다
- ✓ 판별 10회 테스트 — 몇 승 몇 패?

💡 포인트

- 💡 판별이 틀리면 **틀린 사례를 보여주며** 지표를 고치게 — 이게 알고리즘 튜닝

이름 규칙이 엉망인 파일 무더기를 규칙대로 정리 — 손으로 1시간, 시키면 1분.

📄 프롬프트 ① — 난장판 만들기

복사

연습용 폴더 messy_data를 만들고 가짜 측정 파일 100개를 만들어줘. 파일명 규칙이 제각각이게: "LOT123_20260701.csv", "20260702-lot124.csv", "측정결과_lot125(final).csv" 같은 식으로 뒤죽박죽. 내용은 아무 숫자 몇 줄이면 돼.

📄 프롬프트 ② — 정리 (계획 먼저!)

복사

messy_data의 파일들을 "LOT번호_날짜.csv" (예: LOT123_2026-07-01.csv) 형식으로 통일해서 clean_data 폴더로 정리하려고 해. 먼저 변경 계획표만 보여줘. 아직 실행하지 마. 원본은 건드리지 말고 복사본으로만 작업해.

👁️ 예상 결과

측정결과_lot125 (final).csv → LOT125_2026-07-03.csv

20260702-lot124.csv → LOT124_2026-07-02.csv

변경 계획표 100건 → 승인 → 실행 보고: 98 처리 · 2 규칙 불일치 격리

✅ 완료 조건

✓ 계획표를 검토한 후 실행시켰다

✓ 원본 messy_data는 무사하다

💡 안전 수칙

💡 파일 자동화는 ①계획 보기 ②사본 작업 ③결과 보고 3종 세트

부록 E-2 CSV 여러 개 → 주간 리포트 자동 생성

🕒 15분

★★★

"매주 월요일마다 하던 그 취합"을 스크립트 한 방으로.

📁 weekly/ 폴더 (data.zip에 포함)

mon~fri.csv · 장비 6대 가동 데이터

📋 리포트 생성기 프롬프트

복사

weekly 폴더의 CSV(mon~fri)를 전부 읽어 주간 리포트를 만드는 스크립트를 작성해줘.
실행하면 weekly_report.html이 생성되는 것.

1. 장비별 주간 가동률 표 (최악 장비 강조)
2. 일별 에러 추이 차트
3. "특이사항" 자동 요약 문단

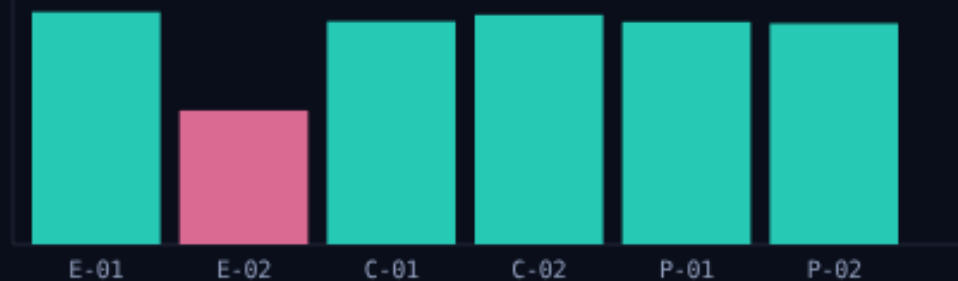
다음 주에 파일만 갈아끼우면 재사용되게. 실행 방법도 알려줘.

🚀 더 해보기

🚀 엑셀(.xlsx) 출력 / 지난주 대비 변화율 컬럼

👁️ 예상 결과

주간 가동률 - ETC-02 78% ▲ (수요일)



✅ 완료 조건

- ✓ 스크립트 실행 → 리포트 생성
- ✓ 숨어 있는 문제 장비·요일이 특이사항에 잡혔다

💡 포인트

💡 도구와 자동화의 차이 = "다음에도 쓸 수 있게"라는 요구

로그 파싱(3-3) + 보고서 문체(2-2) = 매일 아침 10분 은퇴.

📁 tool.log (LAB 3-3과 동일)

📋 시작 프롬프트

[복사](#)

tool.log를 읽어서 "야간 장비 현황 아침 브리핑" 메일 초안을 만드는 스크립트를 작성해줘. 실행하면 briefing.txt가 생성되는 것. 내용: 제목 한 줄 / 밤사이 ERROR 요약(시각·모듈·횟수) / 반복 패턴 경고 / 조치 필요 체크리스트. 팀장님께 보내는 톤으로 5줄 이내.

🚀 더 해보기

🚀 "ERROR 0건인 날은 축하 멘트로" / HTML 메일 버전

👁️ 예상 결과

제목	[야간 브리핑] ERROR 8건 — VACUUM 반복 이슈 확인 필요
요약	02~04시 PG-2 게이지 타임아웃 3회 반복 발생
조치	☐ PG-2 커넥터 점검 ☐ 재발 시 게이지 교체

✅ 완료 조건

- ✓ briefing.txt가 그대로 보내도 될 수준
- ✓ 로그를 바꿔 재실행해도 동작

💡 포인트

- 💡 오늘 배운 것들이 레고처럼 조립됨 — 파싱+요약+문체

엔지니어가 하루 종일 사는 곳 — 엑셀 파일을 읽고, 집계하고, 서식까지 갖춰 다시 저장.

📁 lot_summary.xlsx 받기

raw 시트 60행 + 빈 summary 시트

📋 시작 프롬프트

[복사](#)

lot_summary.xlsx의 raw 시트를 읽어서 summary 시트를 채워줘. 제품(product)별로
①랏 수 ②평균 수율 ③평균 CD ④총 결함 수를 집계한 표로. 수율이 가장 낮은 제품은
셀을 빨강게 강조하고, 헤더는 굵게. 결과를 lot_summary_done.xlsx로 저장해줘.

📋 이어서 — 재사용 가능하게

[복사](#)

다음 달 파일도 같은 형식이면 그대로 돌아가게 스크립트로 만들어줘. 실행 방법이란, raw
시트 컬럼이 바뀌면 어디를 고치면 되는지도 알려줘.

🚀 더 해보기

🚀 제품별 수율 막대 차트를 엑셀 안에 삽입 / 피벗 테이블로

👁️ 예상 결과

📊 summary 시트

제품	랏	평균수율	결함계
A7	16	92.0	78
B3	19	91.9	96
C1	25	90.0 ▲	171

최저 수율 C1 셀 빨강 강조 · 헤더 굵게

✅ 완료 조건

- ✅ summary 시트가 채워진 .xlsx 가 생성됐다
- ✅ 엑셀에서 열어 서식(색·굵게)이 살아있다
- ✅ 최저 수율 C1이 잡혔다

💡 포인트

💡 AI는 .xlsx를 직접 읽고 씁니다 — CSV로 바꿀 필요 없음

문서 여러 개를 지식 기반 삼아 답하기 — 문서 사이 모순도 하나 숨어 있습니다.

📁 specs/ 폴더 (data.zip에 포함)

식각·증착·안전 스펙 3종 (가상)

📋 질문 공세 프롬프트

복사

specs 폴더 문서들만 근거로 답해줘. 근거 문서명과 해당 구절을 반드시 인용하고, 문서에 없으면 "문서에 없음"이라고 답해.

Q1. 식각 공정 압력 상한과 그 근거는?

Q2. 인터락이 걸리는 조건을 전부 나열해줘.

Q3. 문서들 사이에 서로 모순되는 내용이 있는지 찾아줘.

🚀 더 해보기

🚀 "신입용 요약 카드 한 장으로 만들어줘"

👁️ 예상 결과

Q3. 모순되는 내용 있어?

있습니다. 공정 중단 압력이 120 mTorr인데 안전 인터락은 100 mTorr에서 이미 작동합니다. — etch_spec.md §3 vs safety_spec.md §3

✅ 완료 조건

✓ 답변마다 출처 인용이 붙는다

✓ 숨어 있는 모순을 찾아냈다

💡 포인트

💡 문서 간 모순 찾기 — 사람이 제일 못하고 AI가 제일 잘하는 일

매주 쓰는 주간 보고서 프롬프트를 `/weekly-report` 스킬로. 파일은 Claude Code에게 만들어달라고 하면 됩니다.

🤖 ① CLAUDE CODE에게 스킬로 만들어달라 하기

복사

오늘 내가 여러 번 쓴 이 프롬프트를 재사용할 수 있게 스킬로 만들어줘.

`.claude/skills/weekly-report/SKILL.md` 로 저장하고, description에는 "주간 보고 요청 시 자동으로 불러야 한다"는 걸 구체적으로 써줘.

프롬프트: 이번 주 한 일 목록을 주면 ①3줄 요약 ②주요 성과 ③다음 주 계획 형식의 한국어 주간 보고서로 정리해줘.

📄 ② 만들어질 SKILL.MD (예시)

복사

```
---
name: weekly-report
description: 주간 보고서 초안 생성. 이번 주 한 일 목록을 주면 3줄 요약·성과·다음 주 계획으로 정리. "주간 보고", "이번 주 리포트" 요청 시 사용.
---
```

하는 일

아래 형식으로 한국어 주간 보고서를 작성한다.

1. 3줄 요약
2. 주요 성과 (불릿)
3. 다음 주 계획 (불릿)

정보가 부족하면 먼저 되묻는다.

⚙️ ③ 확인하고 호출

- 💡 `/skills` 로 목록에 댄는지 확인
- 💡 새 대화에서 `/weekly-report` 입력 — 또는 "주간보고 써줘"라고만 해도 **자동 호출**
- 💡 `.claude/skills/` 를 **git에 커밋**하면 팀 전체가 같은 스킬 사용

✅ 완료 조건

- ✅ SKILL.md가 생겼고 `/skills` 에 뜬다
- ✅ `/weekly-report` 가 실제로 동작한다

🚀 한 단계 더 — SKILL-CREATOR

- 🚀 스킬을 **평가·튜닝**해주는 플러그인. `/plugin install skill-creator@claude-plugins-official` → `/reload-plugins`
- 🚀 "skill-creator로 내 weekly-report 스킬 평가해줘" — 테스트·벤치마크로 description을 다듬어 줍니다

ADVANCED TRACK · 심화

진짜 앱을 만들어 봅니다

여기부터는 "장난감"이 아니라 실제로 배포해서 팀이 쓰는 도구입니다.

다운로드한 데이터로 대시보드, 브라우저 안의 DB와 CRUD 앱, 웨이퍼 비교기, 그리고 GitHub Pages 배포까지.

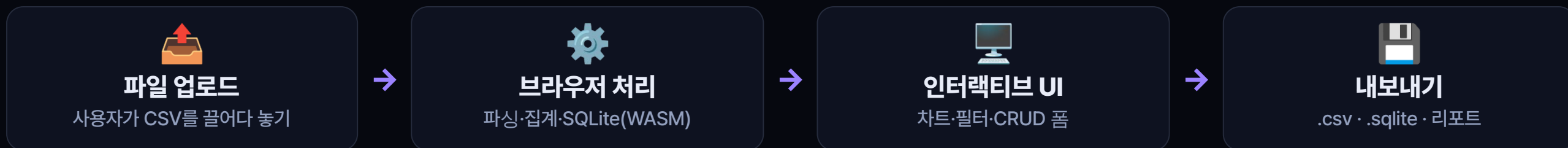
🎯 파일 업로드 → 완성형 웹앱 → 공개 URL

✍️ 심화 실습 3개 (★★★) + 배포

🕒 빠른 그룹 / 데모용



업로드 → 처리 → 저장, 백엔드 없이 브라우저에서



- 서버가 없어도 됩니다 — **HTML 파일 하나가 곧 앱**. 사내망 공유도, USB 전달도 쉬움.
- 데이터가 **내 브라우저 밖으로 안 나갑니다** — 보안 관점에서도 유리 (그래도 오늘은 합성 데이터).

CSV를 끌어다 놓으면 살아나는 대시보드. 800여 건 런 로그에서 문제 장비를 찾아냅니다.

[📁 fab_runs.csv 받기](#)

2주 · 824런 · tool/product/recipe/result/defect

📋 프롬프트 ① — 골격

[복사](#)

fab_runs.csv를 드래그&드롭으로 업로드하면 분석해주는 대시보드를 fab.html 파일 하나로 만들어줘. 라이브러리는 CDN으로 불러와도 돼. 먼저 업로드 영역 + CSV 파싱 + "상단 KPI 카드(총 런 수, 전체 fail율, 최다 fail 장비)"까지만. 다른 건 아직 만들지 마.

📋 프롬프트 ② → ③ — 쌓기

[복사](#)

② 장비별 fail율 막대 차트를 추가하고, fail율이 높은 장비는 빨강계. 이상 장비가 있으면 한 줄로 짚어줘.

③ 날짜 범위 슬라이더와 product 필터를 추가하고, 필터를 바꾸면 모든 차트가 같이 갱신되게. 장비를 클릭하면 그 장비의 런만 표로 드릴다운.

🚀 더 해보기

🚀 "필터 상태를 URL에 저장해서 링크 공유되게" / PNG로 내보내기

👁️ 예상 결과



✅ 완료 조건

- ✓ 파일을 끌어다 놓으면 KPI·차트가 뜬다
- ✓ 필터·드릴다운이 동작한다
- ✓ 숨은 **문제 장비를 찾았다** (fail율 급등)

💡 포인트

- 💡 골격 → 차트 → 인터랙션 **순서대로** 쌓기 (한 방에 X)

브라우저 안에 진짜 SQLite DB를 띄우고, 설비 자산을 CRUD하는 SPA를 만듭니다.

📁 pm_assets.csv 받기

설비 12대 · PM 주기/상태 시드 데이터

📋 프롬프트 ① — DB 구축

[복사](#)

브라우저에서 도는 SQLite로 설비 자산 관리 앱을 assets.html 파일 하나로 만들려고 해. sql.js(SQLite WASM)를 CDN에서 불러와. 먼저 pm_assets.csv를 업로드하면 assets 테이블을 만들어 넣고, 목록을 표로 보여주는 것까지. 컬럼: asset_id, name, type, location, install_date, last_pm, pm_interval_days, status.

📋 프롬프트 ② — CRUD + 지속성

[복사](#)

이제 CRUD를 완성해줘.

1. 추가/수정/삭제 폼 (수정은 행 클릭)
2. type-status 필터와 이름 검색
3. last_pm + pm_interval_days로 "다음 PM 예정일"을 계산해 보여주고, 지난 건 빨강게
4. DB를 .sqlite 파일로 내보내기 / 다시 불러오기
5. 새로고침해도 유지되게 (localStorage)

🚀 더 해보기

🚀 SQL 직접 입력 콘솔 / PM 이력 테이블을 조인해 관리

👁️ 예상 결과

[+ 추가](#)

검색



type



status

설비	TYPE	다음 PM	상태
ETCH-01	식각	07-16	RUN
IMP-01	이온주입	06-14 지남	DOWN
METRO-01	계측	07-28	RUN



.sqlite 내보내기 · localStorage 유지

✅ 완료 조건

- ✓ 추가·수정·삭제가 표에 즉시 반영
- ✓ 지난 PM이 빨강게 뜬다
- ✓ 새로고침해도 데이터가 남는다

🚚 막혔다면

- 💡 sql.js 로딩 실패 → "WASM 경로/CDN 에러를 콘솔에서 확인하고 고쳐줘"

두 웨이퍼를 겹쳐 새로 생긴 불량만 도드라지게. 공정 전후 비교의 정석.

📁 wafermap_A.csv

📁 wafermap_B.csv

A→B: 무엇이 달라졌나?

📋 시작 프롬프트

복사

웨이퍼 맵 두 개를 비교하는 diff 뷰어를 wdiff.html 파일 하나로 만들어줘.

wafermap_A.csv(before)와 wafermap_B.csv(after)를 각각 업로드하면:

1. 왼쪽에 A, 오른쪽에 B 웨이퍼 맵
2. 가운데에 diff 맵 — B에서 새로 불량이 된 die는 빨강, 회복된 die는 초록, 나머지는 흐리게
3. 새 불량 개수와 위치, 그리고 "패턴이 있으면" 한 줄 코멘트
4. 슬라이더로 A↔B 오버레이 페이드

🚀 더 해보기

🚀 여러 스텝(A→B→C) 연속 비교 / 새 불량 die 목록 CSV 내보내기

👁️ 예상 결과



✅ 완료 조건

- ✓ 세 개 맵(A · diff · B)이 나란히 뜬다
- ✓ B에 새로 생긴 불량 패턴을 찾았다 — 무슨 모양?

💡 포인트

💡 "before/after 비교"는 계측·수율·불량 어디에나 재사용되는 패턴

만든 앱, 링크 하나로 — 무료로 웹에 올리기

방금 만든 `index.html` 을 내 GitHub 계정 레포에 올리면, 서버도 비용도 없이 `https://아이디.github.io/레포이름/` 주소가 생깁니다. 팀에게는 링크만 보내면 끝.



① 레포 생성

내 계정에 새 저장소 (Public)



② 파일 push

`index.html`을 main 브랜치로



③ Pages 켜기

Settings ▸ Pages ▸ main ▸ /root



④ 공개 URL

1~2분 뒤 접속 가능

- 배포 방식은 "**Deploy from a branch**" — **main** 브랜치의 루트(/)를 그대로 정적 서빙합니다. 빌드 서버도, 파이프라인도 없이 파일 그대로.
- 한 번 켜두면 이후로는 **push만 하면 자동 반영**. HTTPS도 자동입니다.
-  단, Public 레포는 **전 세계에 공개**됩니다 — 오늘처럼 **합성/더미 데이터**만. 사내 데이터는 절대 올리지 않기.

A-1에서 만든 `fab.html` 대시보드를 진짜 URL로. main 브랜치 루트를 그대로 서빙합니다.

📄 ① 파일 준비 + 올리기 (터미널)

복사

```
# Pages는 루트의 index.html을 먼저 얹 → 진입 파일 이름을 맞춰준다
cp fab.html index.html
```

```
git init
git add index.html
git commit -m "Add fab dashboard"
git branch -M main
```

```
# 아래 주소는 방금 GitHub에서 만든 레포로 바꾸기
git remote add origin https://github.com/<아이디>/fab-dashboard.git
git push -u origin main
```

🤖 또는 CLAUDE CODE에게 통째로

복사

이 폴더를 GitHub Pages로 배포하고 싶어. fab.html을 index.html로 복사하고, git 저장소로 초기화해서 커밋해줘. 그다음 GitHub 웹에서 새 레포(Public)를 만들고 Settings에서 Pages를 켜는 방법을 단계별로 알려줘.

⚙️ ② 레포 만들고 PAGES 켜기 (웹)

💡 github.com ▶ **New repository** ▶ 이름 `fab-dashboard` ▶ **Public** ▶ Create (README 없이)

💡 레포 ▶ **Settings** ▶ **Pages**

💡 Source: **Deploy from a branch**

💡 Branch: **main** · 폴더: **/ (root)** ▶ **Save**

✅ 완료 조건

✅ `https://아이디.github.io/fab-dashboard/` 접속 시 대시보드가 뜬다

✅ 파일을 고쳐 **다시 push**하면 URL에 반영된다

SPA를 올릴 때 자주 걸리는 것들

화면이 깨지거나 새로고침에서 404가 뜬다면 십중팔구 아래 셋 중 하나입니다.

🚫 3대 합정

- 💡 **진입 파일** — 루트에 `index.html` 이 없으면 빈 화면/404. 진입 파일은 반드시 이 이름.
- 💡 **절대경로** — 앱이 `/레포이름/` 하위에서 열립니다. `/style.css` (❌) → `./style.css` (✅). Vite·React 빌드면 `base: '/레포이름/'`.
- 💡 **새로고침 404** — 클라이언트 라우터(`/page` 직접 접속)는 404. `index.html` 을 `404.html` 로 복사하거나 HashRouter 사용.

✅ 배포 체크리스트

- ✓ 루트에 빈 `.nojekyll` 파일 (`_` 로 시작하는 폴더 무시 방지)
- ✓ 안 바뀌어 보이면 강력 새로고침 `Ctrl+Shift+R` · 첫 배포는 1~2분
- ✓ 🔒 Public = 전 세계 공개 → 합성/더미 데이터만

🚚 막히면 이 프롬프트

복사

GitHub Pages(<https://아이디.github.io/레포이름/>)에 올렸는데 화면이 깨지거나 새로고침하면 404가 떠. 원인을 찾아서 진입 파일(index.html)·자원 상대경로·base 경로·404.html 문제를 점검하고 고쳐줘.

방금 만든 것들, 사실 대단한 겁니다



업로드 대시보드

BI 툴 없이 CSV → 인터랙티브 분석. 배포도 공유도 파일 하나.



SQLite CRUD 앱

엑셀 대장을 대체하는 진짜 DB 앱. 서버·설치 0.



비교 뷰어

before/after 패턴 — 계측·수율·불량에 두루 재사용.

...게다가 방금 **GitHub Pages 링크 하나**로 팀에 배포까지 해봤습니다.

몇 달 전이라면 **개발팀에 요청서를 넣고 몇 주**를 기다렸을 것들 — 이제는 오후 한나절. 이게 바이브 코딩의 레버리지입니다.